Understanding Your Structural Learning System

A Plain-Language Guide to the Architecture, Design and Algorithms

A self-organizing, thermodynamically inspired concept-learning framework built from an encoder, an adaptive Gaussian cluster layer, a temperature-driven structural mutation engine, and a growing long-term concept memory.

Prepared as a technical walkthrough of the codebase, written for a general audience.

Table of Contents

1. Executive Summary

This codebase implements a system that learns to organize raw data into meaningful categories entirely on its own, without being told in advance how many categories exist or what they should look like. It does this continuously, one small batch of data at a time, and it is designed to keep reorganizing itself forever — merging categories that turn out to be duplicates, splitting categories that turn out to be hiding two different ideas, forgetting categories that stop being useful, and gradually promoting the most stable categories into permanent, named "concepts" that persist even if the raw data clusters underneath them are later reshuffled.

The design borrows two big ideas from outside pure machine learning. The first is **thermodynamics**: every region of the system has a "temperature" that rises when things are unstable or ambiguous and falls when things settle down, and that temperature directly controls how much structural change is allowed to happen. The second is **biological memory consolidation**: short-term, noisy, per-example memories ("episodic memory") are gradually distilled into long-term, stable, abstract "concepts" — the same way a brain is thought to consolidate a day's experiences into durable knowledge while sleeping.

The system is composed of roughly eighteen cooperating Python modules. None of them do anything very complicated in isolation — mostly averaging, distance measurements, and simple bookkeeping — but the way they are wired together produces a genuinely adaptive, self-restructuring learning loop. This document walks through every module, explains what it does and why, and then walks through exactly what happens, in order, during a single call to the system's main **step()** function, which is the heartbeat of the whole framework.

The last two sections are worth reading even if you skip the deep dives: Section 20 explains the overall design philosophy in plain terms, and Section 21 is an honest list of rough edges — leftover debug prints, dead/commented-out code, and a few places where the logic is duplicated or slightly inconsistent — that you will want to clean up before treating this as production-grade.

2. The Big Picture — What Problem Is This Solving?

Most machine learning systems are told in advance how many categories to expect ("there are 10 classes"), and their internal structure is fixed once training finishes. This system is built for the opposite situation: a stream of data arrives continuously, nobody knows ahead of time how many natural categories it contains, and the number of categories may need to grow or shrink over time as more data is seen.

> Think of it like a librarian with no catalogue system, who receives an endless stream of unlabeled documents and has to invent, on the fly, a set of shelves and labels that make sense — merging two shelves that turned out to hold the same topic, splitting a shelf that has become an overstuffed mix of two topics, and eventually writing permanent index cards ("concepts") for the topics that keep proving themselves useful over time.

Concretely, the system takes raw input vectors, encodes them into a compact embedding space, assigns each one (softly, i.e. with probabilities, not a hard yes/no) to one of a changing number of Gaussian "clusters", and continuously reshapes that population of clusters — through a slow, temperature-gated process of merging, splitting, birth, and death — while a second, longer-memory layer above the clusters gradually crystallizes the most persistent clusters into durable "concepts" connected in a graph of relationships.

3. System at a Glance — The Component Map

The table below lists every file in the project, what role it plays, and which layer of the system it belongs to. Three layers are useful to keep in mind while reading the rest of this document:

- **Perception layer** — turns raw input into vectors (encoder.py).

- **Structural / clustering layer** — the fast-moving population of Gaussian clusters, their temperature, redundancy tracking, and the merge/split machinery that reshapes them (cluster.py, temperature.py, redundancy.py, structural_queue.py, budget_scheduler.py, structural_executor.py, structural_pressure.py, cluster_persistence.py).
- **Semantic / concept layer** — the slower-moving, longer-memory layer that turns stable clusters into permanent concepts and relates them to each other (memory_store.py, consolidation.py, concept_memory.py, concept_field.py, micro_cluster.py, concept_graph.py, concept_reasoner.py).

File	Role	Layer
encoder.py	Neural network that turns raw input into an embedding vector.	Perception
cluster.py	The population of Gaussian clusters: assignment + EM learning.	Structural
cluster_persistence.py	Saves/loads the cluster population to/from disk.	Structural
temperature.py	Per-cluster "excitement level" that gates structural change.	Structural
redundancy.py	Tracks how similar every pair of clusters is, over time.	Structural
structural_pressure.py	Detects sustained low confidence / high distance as a warning signal.	Structural
structural_queue.py	A priority queue of proposed merge/split events.	Structural
budget_scheduler.py	Decides how many structural changes are allowed this cycle.	Structural
structural_executor.py	Actually performs merges/splits, with an energy-based accept/reject test.	Structural
memory_store.py	Episodic memory: stores representative examples per cluster.	Concept
consolidation.py	Decides which clusters are "stable enough" to consolidate.	Concept
concept_memory.py	Permanent concept store; concept creation and lifecycle events.	Concept
concept_field.py	Tracks the fine internal shape of a concept as new examples arrive.	Concept
micro_cluster.py	Small sub-clusters inside a concept field's "grey zone".	Concept
concept_graph.py	A weighted graph of relationships between concepts.	Concept
concept_reasoner.py	Spreading-activation search over the concept graph.	Concept
energy_controller.py	Combines several signals into one "is the system healthy" score.	Cross-cutting

File	Role	Layer
trace_engine.py	Records the most recent decision for debugging/explanation.	Cross-cutting
structural_system.py	The orchestrator — owns everything and runs the training step.	Orchestrator
debug_dataset.py	A standalone utility script to download a sample text dataset.	Tooling

4. Walking Through One Training Step

The best way to understand the whole system is to follow exactly what happens, in order, every time **StructuralSystem.step(x)** is called with one batch of raw input **x**. This function is the heartbeat of the framework — everything else exists to support one of the sub-steps below.

Step 1 — Encode

The raw batch is passed through the encoder to produce embedding vectors **z**. The vectors are then L2-normalized (rescaled to unit length, so only their direction matters, not their magnitude) and a tiny amount of random noise is added before re-normalizing. This noise injection is deliberate — it stops the system from getting permanently "stuck" if a batch of inputs happens to be identical or nearly so.

Step 2 — Cluster Assignment

Each vector is compared against every existing cluster using a Mahalanobis distance (a distance measure that accounts for each cluster's own shape/spread, not just raw Euclidean distance) and converted into a soft probability distribution over clusters — i.e. "70% cluster 3, 20% cluster 7, 10% everything else" rather than a single hard label. The temperature used to sharpen or soften this distribution is itself computed adaptively from how uncertain the raw distances already are, which is explained in Section 6.

Step 3 — EM Update (Learning)

Using those soft assignments, every cluster's centre, spread, and "mass" (how much data it represents) are nudged toward the batch, using a sharpened version of the probabilities so that confident points pull harder than ambiguous ones. Dead clusters (ones whose mass has collapsed far below average) are revived by re-seeding them near a random data point, which prevents the population from silently losing capacity over time.

Step 4 — Global Energy

The system computes redundancy across all cluster pairs, estimates "memory pressure" (how full the episodic memory is relative to its total capacity), and estimates "concept coherence" from the current concept store, then feeds all of this into the Energy Controller (Section 17) to get one overall health number for the whole system at this moment.

Step 5 — Episodic Memory Write

The batch's vectors, their assigned clusters, and confidence scores are written into the Memory Store (Section 11), which keeps a rolling, confidence-weighted, duplicate-filtered sample of recent examples for every cluster. This is the raw material that later gets distilled into permanent concepts.

Step 6 — Concept Formation Loop

A random subsample of up to 200 points from the batch is checked against all existing concepts. Points that don't resemble any known concept closely enough (the similarity bar rises as the concept store grows, from 0.85 up to 0.95) spawn a brand-new concept; points that do resemble an existing concept reinforce it. The number of new concepts allowed per step is itself capped adaptively — the more uncertain (higher-entropy) the clustering currently is, the more new concepts are allowed to form, up to a ceiling.

Step 7 — Drift & Flow

Two simple diagnostics are computed: **drift** (how far each cluster's centre moved since the last update — a proxy for instability) and **flow** (the raw magnitude of each cluster's centre — a proxy for how "active" it currently is).

Step 8 — Temperature Update

Entropy, drift, flow and memory pressure are combined into a differential-equation-style update of each cluster's temperature (Section 7). Rising instability heats a cluster up; settling down cools it back down.

Step 9 — Redundancy Update

The pairwise similarity between every pair of cluster centres is recomputed and smoothly accumulated into a persistent redundancy matrix (Section 8), so that a pair of clusters that keeps looking similar over many steps builds up a strong, lasting "these two might be the same thing" signal, rather than reacting to one noisy moment.

Step 10 — Structural Proposals

Using the freshly updated redundancy matrix and temperature-derived split signal, candidate merge and split events are pushed onto the Structural Queue (Section 9) with a priority score. This step only *proposes* changes — nothing is actually merged or split yet.

Step 11 — Structural Execution (every N steps)

Every **structural_interval** steps (50 by default), the system enters its slow-timescale phase: the Budget Scheduler decides how many structural operations are affordable right now, and the Structural Executor pops proposals off the queue and tries them one at a time, accepting or rejecting each one based on whether it improves a cheap energy estimate (Section 10). Afterwards, any cluster whose mass has collapsed to near zero is removed outright, and the Consolidation Controller (Section 12) checks which surviving clusters are stable enough to be linked into permanent concepts, after which the Concept Graph (Section 15) is refreshed.

Step 12 — Bookkeeping

Finally, the batch size is recorded by the Budget Scheduler (so its budget calculation, which grows logarithmically with total data seen, stays accurate), and a dictionary of statistics — entropy, cluster count, temperature, concept count, and more — is returned to the caller.

5. Deep Dive: The Perception Layer (Encoder)

File: encoder.py — class AdvancedEncoder

The encoder's job is simply to turn a raw input vector into a smaller, better-organized "embedding" vector that the rest of the system can reason about. It is a fairly standard modern deep network: an input projection, a stack of residual blocks (eight by default), and a final head that produces the embedding.

- **Spectral normalization** is applied to every linear layer. This caps how much any single layer can "stretch" its input, which keeps the whole network numerically well behaved (Lipschitz-stable) — important because unstable embeddings would make every downstream distance calculation unreliable.
- **GEGLU feed-forward blocks** (a gated variant of the now-common "GLU" feed-forward design used in modern transformers) give the network a learned gating mechanism rather than a plain non-linearity.
- **Residual + gated blocks**: each block adds its own transformation back onto its input, but the amount added is itself controlled by a learned sigmoid gate, so the network can choose to pass information through mostly unchanged if that is safer.
- **Variational head (optional)**: instead of outputting a single deterministic vector, the head can output a small Gaussian (mean + log-variance) and sample from it. This adds a controlled amount of randomness to embeddings, which acts as a mild regularizer.
- **Learnable temperature scaling**: a single learned scalar rescales the final embedding, bounded so it can't grow or shrink without limit.

In short: this file is a conventional, carefully stabilized embedding network. Nothing about it is unique to the structural-learning ideas elsewhere in the project — its only job is to hand the rest of the system clean, well-scaled vectors to work with.

6. Deep Dive: The Cluster Layer — Regions of Meaning

File: cluster.py — class GaussianClusterLayer

This is the structural heart of the system. It maintains a changing population of "clusters", each one a soft Gaussian blob defined by three learned quantities: a centre (**mu**), a spread along each dimension (**log_sigma**, stored in log form for numerical stability), and a "mass" representing how much of the data it currently represents.

> Think of each cluster as a fuzzy, glowing region floating in the embedding space. New data points land closer to some glowing regions than others; the regions themselves slowly drift, brighten, dim, split apart, or merge together as more data arrives — nobody hand-designed how many regions there should be.

6.1 Assigning a point to clusters

Assignment is not a hard "this point belongs to cluster 3" decision. Instead, **assign()** computes a Mahalanobis distance from the point to every cluster centre (a distance that stretches or compresses each axis according to that cluster's own learned spread, so a cluster that is naturally wide in one direction doesn't unfairly penalize points that land far along that direction), combines that with each cluster's log-probability (a soft-max over cluster "mass"), and turns the result into a probability distribution over all clusters.

A distinctive feature here is **adaptive temperature**: before producing the final probabilities, the code first computes a rough baseline distribution, measures how uncertain (high-entropy) that baseline already is relative to the maximum possible uncertainty for the current number of clusters, and uses that to pick a sharpening temperature between 0.2 and 0.7. In plain terms — if the raw distances already point clearly to one cluster, the system sharpens gently; if the raw distances are ambiguous, it sharpens more aggressively so the final decision doesn't stay mushy forever.

6.2 Learning (the EM update)

The **update()** method is a hardened version of the classic Expectation-Maximization algorithm used to fit Gaussian mixtures. A few notable design choices:

- **Sharpened responsibilities**: before computing weighted averages, the soft assignment probabilities are squared and renormalized. This makes confident points count more and ambiguous points count less when updating a cluster's centre — a soft version of "only listen strongly to the points that are clearly yours".
- **Dead-cluster prevention**: a small amount of random noise is added to every cluster centre after each update, purely to break exact symmetry and stop clusters from freezing in place.
- **Mass renormalization**: after the update, all cluster masses are rescaled to sum to one, so "mass" is always interpretable as a share of the total population, not an unbounded running count.
- **Rebirth of dead clusters**: any cluster whose mass falls below half the average mass is immediately "reborn" — its centre is reset to a random data point from the current batch (plus a little noise) and its mass is reset to the average. This is the mechanism that keeps a fixed-size cluster population from wasting capacity on clusters nobody uses any more.
- **Variance and mass clamps**: spreads are clamped between a configured minimum and maximum variance so no cluster can collapse to a single point or balloon without bound.

Several other mechanisms exist in the file only as commented-out code — a repulsion force to physically push cluster centres apart, a "semantic gravity" pull toward the memory store's remembered cluster centres, a dynamic cluster-splitting routine triggered by local instability, and a dead-cluster removal routine. None of these are currently active; they represent extra ideas that were tried and left in place for possible future use (see Section 21).

6.3 Initialization

init_from_data() uses k-means++ style sampling: it repeatedly picks the data point that is farthest (in squared distance) from all centres chosen so far, which spreads the initial clusters out sensibly instead of starting them all in one place. It's meant to be called once, before training starts — and skipped if a saved cluster state was already loaded from disk.

6.4 Diagnostics the cluster layer exposes

- **compute_drift()** — how far each cluster centre moved since the previous step.
- **compute_flow()** — the raw magnitude (distance from the origin) of each cluster centre.
- **redundancy_matrix()** — a quick pairwise similarity matrix based on centre distance (a simpler, one-shot cousin of the persistent redundancy tracked in redundancy.py).
- **cluster_statistics()** — mean/max spread, mean mass, and current cluster count.

File: cluster_persistence.py simply saves the three learned tensors (centres, log-spread, mass) to a compressed .npz file and reloads them later, restoring them onto whatever device the cluster layer currently lives on.

7. Deep Dive: Temperature — The System's Excitement Level

File: temperature.py — class TemperatureField

Every cluster has its own scalar "temperature", updated according to a small differential-equation-style rule: temperature rises with entropy (uncertainty in assignments), rises with drift (how much the cluster is currently moving), rises with flow (how far the cluster sits from the origin), and falls due to a constant cooling term plus an extra penalty proportional to the square of memory pressure. The temperature is clamped between 0.1 and a configurable maximum (10.0 by default) so it never runs away in either direction.

> It helps to think of temperature the way you'd think of the temperature of a pot of water on a stove: heat (uncertainty, movement, crowding) makes the water boil and easier to reshape; cooling settles it back down. The rest of the system uses this temperature as a gate — hot clusters are allowed to split or reorganize; cold, settled clusters are left alone and, eventually, promoted to permanent concepts.

Two derived signals come out of temperature:

- **split_signal(entropy)** — combines a threshold-based "boiling over" term (any temperature above 3.0 contributes to a split score) with an entropy-driven bonus, producing a per-cluster score used later to propose splits.
- **merge_modifier()** — a sigmoid that is close to 1 for cold clusters and close to 0 for hot ones, used to damp down merge proposals for clusters that are still actively unstable (merging two things that are still moving around is riskier than merging two things that have settled).

plasticity() converts temperature into a 0–1 "how easy is this cluster to change right now" score via a sigmoid, and **resize()** grows or shrinks the temperature vector whenever the number of clusters changes, carrying over the old values for clusters that survive.

8. Deep Dive: Redundancy — Spotting Duplicate Regions

File: redundancy.py — class RedundancyAccumulator

Where the cluster layer's own **redundancy_matrix()** gives a one-shot, memoryless snapshot of how similar cluster centres currently are, this module keeps a **persistent, slowly accumulating** record of similarity, governed by a simple leaky update: on every call, each pair's stored redundancy value moves toward its instantaneous Gaussian similarity, with a decay term (**kappa**) constantly pulling it back toward zero. This means two clusters have to look alike *consistently, over many steps* before the system treats them as strongly redundant — a single noisy coincidence won't trigger a merge.

After every update the whole matrix is clamped to a maximum value and then rescaled so its largest entry is exactly 1, and it is explicitly symmetrized (forced to be identical whether read as "cluster i vs j" or "cluster j vs i"). **merge_scores()** optionally combines this redundancy with the temperature field's merge modifier, so hot (unstable) cluster pairs get their merge score damped down even if their centres happen to be close together.

resize() and **reset_pair()** keep the matrix in sync whenever the cluster population changes size or a specific pair has just been merged.

9. Deep Dive: The Structural Queue and Budget Scheduler

File: structural_queue.py — class StructuralQueue

This is a priority queue (a max-heap, implemented with Python's **heapq** by storing negative scores) that holds proposed structural events — merges, splits, or new-cluster additions — each tagged with a score. A few safety features make it robust to being used continuously over a long-running process:

- **Duplicate prevention**: each proposal is given a stable key (e.g. the sorted pair of cluster indices for a merge) and the queue refuses to add a second copy of an already-pending proposal.
- **Time-decayed priority**: every time an event is popped, its score is recomputed with an exponential decay based on how long it has been sitting in the queue, so stale proposals naturally lose priority instead of blocking fresher ones forever.
- **Threshold filtering**: proposals below a minimum score are simply never pushed (or are dropped when popped, after decay).
- **Node validation on pop**: when popping, the caller can supply the current set of valid cluster indices, so a proposal referencing a cluster that has since been removed or merged away is silently skipped instead of causing an error.

File: budget_scheduler.py — class BudgetScheduler

Even if the queue is full of good proposals, the system deliberately restricts how many structural changes are allowed per structural phase — this is the "budget". The budget formula combines several ingredients:

- A base budget that grows with the logarithm of total samples seen so far (more experience → a slightly larger budget, but with strongly diminishing returns).
- A memory-pressure modifier that shrinks the budget as the cluster population approaches its configured memory ceiling, so the system doesn't keep expanding forever.
- A temperature modifier that increases the budget when the system is, on average, "hot" (more instability → more restructuring is allowed).
- A cycle modifier that slowly shrinks the budget the more structural cycles have already happened, gently encouraging long-term stabilization.
- A hard floor: if the temperature modifier drops below a stability threshold, the budget is forced down to a small fixed fraction of the maximum, effectively freezing structure when the system is very cold and settled.

The scheduler also exposes **structural_mode()**, which labels the current regime as "expansion" (mean temperature above 1.0), "compression" (below 0.6), or "neutral" — used later by the executor to bias its acceptance behaviour.

10. Deep Dive: The Structural Executor — Merge, Split, Accept, Reject

File: structural_executor.py — class StructuralExecutor

This is where proposed structural changes actually get carried out — or rejected. It is the most elaborate module in the codebase, and it is worth understanding in some detail because it implements a small simulated-annealing-style search rather than blindly applying every proposal.

10.1 Merge

Merging two clusters computes a mass-weighted average of their centres and spreads, removes one of the two clusters outright, and installs the combined statistics into the surviving cluster's slot. The merged cluster is also deliberately cooled (its temperature halved) since a freshly merged cluster should be treated as newly settled, not newly hot. If concept memory is attached, **concept_memory.on_merge()** is notified so that any concepts pointing at the removed cluster get transferred or merged into the surviving cluster's concepts (Section 13.4).

10.2 Split

Splitting takes one cluster and turns it into two: it perturbs the original centre in a random direction (scaled by that cluster's own spread) to create two new centres — one shifted one way, one shifted the other — each inheriting half of the original mass and the same spread. A split is refused outright if the cluster's mass is already below half the population average, since splitting an already-small cluster would create two even-smaller, likely meaningless ones. After a split, the original slot is heated up slightly (since it just changed) while the new sibling starts at the original's pre-split temperature.

10.3 The execute() loop — energy-gated acceptance

Rather than simply applying every popped proposal, **execute()** runs something close to a simulated-annealing inner loop:

- The scheduler is asked for a budget of allowed operations for this phase; if the budget is zero, nothing happens at all this cycle.
- For every candidate proposal, a snapshot of the entire structural state is taken first, so any change can be perfectly rolled back.
- Several safety gates run before an operation is even attempted — refusing merges that would push the cluster count below a floor tied to how many clusters currently hold meaningful mass, refusing merges during an "expansion" regime while the system is still hot, and, for merges specifically, checking that the two clusters' best-matching concepts (if any) are semantically similar enough (cosine similarity above 0.7) before allowing the merge to proceed at all.
- If the operation succeeds mechanically, a cheap proxy for the system's overall "energy" is recomputed (a weighted combination of assignment entropy, pairwise redundancy, instability from temperature, and a penalty for having too few clusters relative to a target count).
- The change is **kept** if it lowered this energy; it is also sometimes kept even when it slightly raised the energy (a small uphill move), with a probability tied to the current mean temperature — this is the "exploration" behaviour borrowed from simulated annealing, letting the system occasionally accept a change that looks slightly worse in the short term if it might lead somewhere better. Otherwise the snapshot is restored and the attempt is discarded.

This energy-gated accept/reject step is what keeps the merge/split machinery from thrashing — a merge or split is only kept if it plausibly makes the overall system simpler, more coherent, or more stable, not merely because it was next in the queue.

10.4 Removing and adding clusters safely

_remove_cluster() and **_add_cluster()** are careful bookkeeping routines: removing a cluster means deleting its row/column from every tensor that is indexed by cluster id (centres, spreads, mass, temperature, the redundancy matrix) *and* re-indexing every dictionary keyed by cluster id in the memory store, so that cluster index 7 in the episodic memory always still refers to the same cluster after another one earlier in the list has been deleted. This kind of index management is easy to get wrong, and this module is the one place responsible for getting it right consistently.

11. Deep Dive: Memory Store — Episodic Memory Per Cluster

File: memory_store.py — class MemoryStore

This module is the system's short-to-medium-term memory: for every cluster it keeps a capped list (100 items by default) of representative example vectors, each stamped with the training step it was seen at and the confidence the cluster assignment had at that time.

- **Confidence gating with a "young cluster" grace period**: a point is only stored if its assignment confidence clears a threshold — but that threshold is relaxed dramatically (down to 0.01) for clusters younger than 500 steps, so brand-new clusters aren't starved of examples before they've had a chance to stabilize.
- **Near-duplicate filtering**: before storing a new point, it is compared against the cluster's five most recently stored points; if it is more than 0.97 cosine-similar to any of them, it is treated as a duplicate and skipped, keeping the stored sample diverse rather than full of near-identical repeats.
- **Recency- and confidence-weighted cluster centre**: **cluster_center()** does not just average the stored vectors — each stored item is weighted by its original confidence *and* by an exponential decay based on how long ago it was seen (a half-life-style "age / 500" decay), so the computed centre leans toward recent, confident examples.
- **cluster_strength()** averages the confidence of the most recent 20 stored items — a simple, robust measure of "how sure has this cluster been about itself lately".
- **decay_memory()** periodically purges items older than a maximum age, and drops a cluster's memory entirely once it has nothing left — a simple form of forgetting.
- **semantic_gravity()** and **memory_entropy()** provide additional diagnostics: how strongly a given vector is pulled toward a cluster's remembered identity, and how evenly memory is spread across clusters overall.

In the biological-memory analogy this module plays the role of the hippocampus: fast, detailed, per-episode memory that is useful in the short term but is not, by itself, permanent knowledge.

12. Deep Dive: Consolidation — Deciding What Becomes Permanent

File: consolidation.py — class ConsolidationController

Every structural phase, this module decides which clusters are stable enough to be considered for promotion into permanent concepts. Rather than using fixed absolute thresholds, it computes adaptive thresholds from the current state of the system when **adaptive=True** (the default):

- The drift threshold is set to the 40th-percentile drift value across all current clusters (with a small hard floor), so "low drift" always means "among the more stable 40% of clusters right now", regardless of the system's absolute scale.
- The minimum memory requirement scales with the memory store's actual per-cluster capacity (5% of it, or a fixed floor, whichever is larger).
- The minimum mass requirement scales with the total mass currently in the system (2% of total mass, or a fixed floor).

A cluster is judged stable if it satisfies *either* of two conditions: low drift combined with sufficient mass, *or* sufficient memory combined with strength above an adaptive bar. Cluster strength is softly discounted (multiplied by 0.85) once a cluster is older than 200 steps, a mild aging effect. If literally no cluster in the whole population qualifies, the single cluster with the smallest drift is used as a fallback, so the consolidation step never comes back completely empty-handed.

13. Deep Dive: Concept Memory — Long-Term Knowledge

File: concept_memory.py — class ConceptMemory

This is the permanent knowledge store. A "concept" is a dictionary holding a centre vector, a slower-moving "core vector", a running variance estimate, a short list of recent prototype vectors, a strength score, an example count, and a "semantic centre" used for comparisons. Unlike clusters, concepts are not tied one-to-one to a fixed population size — many clusters can point at the same concept, and one cluster can spawn several concepts over its lifetime.

13.1 Two ways concepts are created

There are two separate code paths that create concepts, and it's useful to know both exist:

- **Online, per-step creation**, driven directly from **structural_system.step()**: individual data points from the current batch that don't resemble any existing concept closely enough call **create_new_concept()** immediately, giving the concept an initial strength of 1.0 and a single example.
- **Batch consolidation**, driven from the slower structural phase: **consolidate_cluster()** looks at a whole cluster's accumulated memory (via the memory store), checks it against adaptive strength/size thresholds and a similarity check against existing concepts, and only then creates a new concept seeded from the cluster's confidence-and-recency-weighted centre. This path is more conservative and includes a "grace" mechanism — a cluster is allowed up to five failed attempts (tracked in **cluster_attempts**) before consolidation gives up on it for a while.

13.2 Similarity and identity

similarity() is a straightforward cosine similarity against a concept's semantic centre. **identity_score()** is more interesting: it measures not just how similar a vector is to *this* concept, but the *margin* between that similarity and the best similarity to any *other* concept — in other words, how confidently a point belongs to this concept rather than a neighbouring one, not just how close it happens to be in absolute terms.

13.3 Stabilizing a concept over time

stabilize_concept() updates a concept toward a new observed vector using two nested moving averages: a fast update rate that decreases the more the concept has already been updated (so early observations move the concept a lot, later ones move it only a little — the classic "learning rate that shrinks with experience" pattern), followed by a second, slower blend (90% old "core" value, 10% new) that anchors the concept's centre to its long-term core identity so it can't be dragged too far by any single recent burst of examples.

13.4 Lifecycle: what happens to concepts when clusters split, merge, or die

- **on_split()**: when a cluster splits into two, the original keeps its existing concept mapping and the brand-new sibling cluster starts with no concept at all — it has to earn one naturally through future consolidation.
- **on_merge()**: when two clusters merge, every concept attached to the cluster being removed is compared against every concept attached to the surviving cluster; if a close enough match is found (cosine similarity above 0.97) the two concepts are fused into one (their strengths add, and their centres are blended 70/30 in favour of the survivor); otherwise the orphaned concept is simply reattached to the surviving cluster rather than being deleted.
- **on_death()**: when a cluster is removed outright because its mass collapsed to near zero, every concept that was uniquely attached to it is deleted, and any edges referencing it are stripped out of the concept graph as well.

The file also includes straightforward save/load support (both an .npz format and a pickle fallback) for persisting the concept store across sessions.

14. Deep Dive: Concept Fields and Micro-Clusters

Files: concept_field.py, micro_cluster.py

Once a concept exists, a **ConceptField** tracks its fine internal structure as new vectors keep arriving. Every incoming vector is compared to the field's centre by cosine similarity and sorted into one of three bands:

- **Core (similarity ≥ 0.85)** — the vector is added to a rolling list of up to 50 "core" vectors, and the field's centre is nudged toward their average using an exponential moving average (10% weight on the new information each time).
- **Grey zone (0.48 ≤ similarity < 0.85)** — the vector is neither clearly inside nor clearly outside the concept. It is handed to a **MicroClusterManager**, which tries to match it against small, informal micro-clusters that have quietly formed near the edge of this concept; if it doesn't match any existing micro-cluster closely enough (above 0.82 similarity), a brand-new micro-cluster is started. This is effectively a lightweight, unsupervised radar for "something new might be forming near the boundary of this concept" — useful as an early warning before a full new concept is formally created elsewhere in the system.
- **Outside (similarity < 0.48)** — the vector is simply ignored by this field.

A **ConceptFieldManager** keeps a dictionary of one field per concept id and provides simple create/add helper methods used throughout the rest of the system.

15. Deep Dive: Concept Graph — Relationships Between Ideas

File: concept_graph.py — class ConceptGraph

While concept_memory.py stores *what* each concept is, concept_graph.py stores *how concepts relate to each other* — a weighted, directed graph where an edge from concept A to concept B represents "these two tend to co-occur / resemble each other, and here is how strongly".

- **update()** compares every pair of currently known concepts, weighting their cosine similarity by an estimate of how "stable" each one is (how close its long-term core vector still is to its current, possibly-drifted centre). An edge is only added or strengthened if the similarity clears a threshold that is itself relaxed for less-stable concept pairs (0.55 up to 0.80 depending on stability) — well-established concepts need less evidence to connect than shaky, still-forming ones.
- **Loop suppression**: before adding an edge, a bounded breadth-first search checks whether a path already exists in the reverse direction; if so, the new edge's score is heavily discounted, which discourages the graph from forming tight mutual-reinforcement loops.
- **Temporal reinforcement**: a repeated edge does not simply get overwritten with a fresh average — it is reinforced additively (existing weight plus 15% of the new score, capped at 1.5), so an edge that keeps being observed keeps getting stronger, similar to how the redundancy accumulator behaves for clusters.
- **Automatic meta-concepts**: whenever an edge's weight climbs above 0.85, the system spontaneously creates a new "meta-concept" node representing that pair, with edges connecting the meta-node down to its two children. This is a simple mechanism for forming higher-level abstractions out of two concepts that have proven, over time, to be tightly linked.
- **Per-node mass control**: if a single concept accumulates too much total outgoing edge weight or too many edges, its edges are re-normalized using a power-law reshaping (exponent 1.5) that keeps the strongest connections while pruning away the weakest ones below a small threshold — this keeps any one concept from becoming implausibly connected to everything.

- **decay()** continuously ages every edge down, with an adaptive rate — strong edges decay more slowly than weak ones, and older edges (that have survived many updates) decay a little more slowly still — until an edge falls below a minimum weight and is deleted outright.
- **propagate_activation()** is a simple multi-hop spreading activation function used elsewhere: starting from one concept with strength 1.0, it repeatedly spreads a decayed, weighted fraction of that strength outward along the graph's edges for a fixed number of hops.

16. Deep Dive: Concept Reasoner — Spreading-Activation Search

File: concept_reasoner.py — class ConceptReasoner

This module implements a more careful, best-first version of the same idea as **propagate_activation()** above, using a priority queue (again via Python's **heapq**) instead of simple breadth-first waves. Starting from a chosen concept, it repeatedly expands the currently most-promising still-unvisited (node, depth) pair, decaying the activation score at each hop (by a fixed decay factor of 0.6) and applying an extra penalty (0.5x) to edges that lead back to a node that has already been visited, which discourages the search from looping in circles.

The search is bounded on several sides at once: a maximum depth (4 hops), a minimum activation floor below which a path is simply abandoned, and a cap on how many times any single node can be revisited (3). The result is a dictionary mapping every concept reached to the strongest activation it received along any path, and **top_k()** simply returns the strongest handful of results — effectively "given this concept, what are the most strongly related concepts, considering both direct and indirect connections?", the same kind of question a human word-association or memory-recall exercise answers.

This is a genuine graph-search algorithm (best-first search with a priority queue, visited tracking, and depth/score bounds), and it is worth noting it is the one place in the codebase doing classical, symbolic-style search rather than statistical/numerical computation — it operates purely on the concept graph built by the other modules.

17. Deep Dive: Energy Controller — The Single Health Score

File: energy_controller.py — class EnergyController

This is deliberately the simplest module in the project, and it plays an important conceptual role: it is the one place where every other signal in the system — assignment entropy, cluster redundancy, drift, memory pressure, concept coherence, and causal consistency (currently unused, defaulting to zero) — gets combined into a single scalar "energy" number, using a fixed set of hand-chosen weights. Lower energy is defined as better: less uncertainty, less duplication, less instability, less memory strain, and more coherent, well-formed concepts.

This is separate from, and simpler than, the local energy proxy computed inside the structural executor (Section 10.3) — the executor's proxy is a cheap approximation recomputed many times per structural phase to gate individual merge/split decisions, while this module's **compute()** produces a single "how is the whole system doing right now" number once per training step, primarily for logging and monitoring rather than for making any decision directly (in the current code it is computed but not yet fed back into any control loop).

18. Deep Dive: Trace Engine — Explainability

File: trace_engine.py — class DecisionTraceEngine

The smallest module in the project. On every training step it records the top three clusters (by average probability across the batch) along with which cluster the very first item in the batch was actually assigned to, and simply keeps the single most recent such record. **explain()** returns it. This is a lightweight debugging/explainability hook rather than a full audit trail — it currently only remembers the most recent decision, not a history.

19. The Structural System — The Conductor

File: structural_system.py — class StructuralSystem

This is the orchestrator that owns every other component (the encoder, cluster layer, temperature field, redundancy accumulator, structural queue, budget scheduler, and structural executor are all passed in at construction time, and the memory store, consolidation controller, concept memory, concept field manager, and concept graph are created internally and wired into the components that need them). Its **step()** method is the full sequence described in Section 4, and its two supporting methods are worth calling out specifically:

_generate_structural_proposals()

Builds the merge and split candidates for the structural queue every single step (cheap to compute), using both the temperature-damped redundancy-based merge score and a second, more direct "if raw redundancy is above 0.15, propose a merge anyway (at half strength)" rule that bypasses temperature damping for clearly redundant pairs. Splits come from the temperature field's split signal, plus a fallback rule that forces a split proposal for any cluster holding more than 40% of total mass whenever entropy has collapsed to near zero — a safeguard against one giant, overconfident cluster silently swallowing the whole system.

_structural_phase()

Runs only every **structural_interval** steps (50 by default) and does four things in order: asks the executor to actually work through the queued proposals within budget; identifies and removes any cluster whose mass has collapsed to near zero, carefully re-indexing the memory store and concept mappings so nothing points at a stale cluster id; asks the consolidation controller which surviving clusters are stable enough to be linked up with concept fields; and finally refreshes and decays the concept graph. This is the "slow timescale" of the system — the equivalent of a nightly consolidation pass, run periodically rather than every single step, because reorganizing structure is expensive and only makes sense to do once enough new evidence has accumulated.

20. Design Philosophy and Why It Is Built This Way

Three ideas run through almost every module in this codebase, and recognizing them makes the whole system much easier to reason about:

20.1 Two timescales, like short-term and long-term memory

Every single batch updates the fast layer (cluster assignment, EM update, temperature, redundancy, episodic memory). Only occasionally — every 50 steps by default — does the system pay the much higher cost of actually restructuring itself (merging, splitting, consolidating into concepts). This mirrors how biological memory is thought to work: moment-to-moment experience updates fast, detailed, short-term memory continuously, while the much slower process of turning that experience into stable, generalized long-term knowledge happens periodically, in batches, rather than after every single moment.

20.2 Temperature as a universal "how much change is allowed right now" gate

Instead of a single global "learning rate", almost every structural decision — whether a cluster is allowed to split, whether a merge is allowed to proceed, how large the structural budget is this cycle — is gated by a temperature that is *local* (every cluster has its own) and *self-regulating* (it rises with instability and falls with settling down, following a simple physically-inspired differential equation). This lets different parts of the embedding space change at different rates simultaneously — a newly forming, still-chaotic region of the data can reorganize itself aggressively while a long-settled region stays essentially frozen, without needing any global coordination.

20.3 Nothing is accepted just because it was proposed

At almost every level, proposals are cheap and permissive, but execution is guarded: concepts are proposed liberally but only consolidated after clearing adaptive strength/size bars; merges and splits are proposed liberally but only kept if a cheap energy estimate says they actually helped (or, occasionally, are allowed through anyway as a controlled, temperature-scaled exploration step); redundancy has to be observed persistently, not just once, before it drives a merge. This "propose freely, accept carefully, and allow yourself to be wrong occasionally" pattern is what keeps a system that is constantly reorganizing itself from oscillating or thrashing.

20.4 Self-scaling thresholds instead of fixed magic numbers

Wherever possible, thresholds are computed *relative to the current state of the system* rather than hard-coded — the consolidation controller's drift/memory/mass bars, the concept-similarity bar that tightens as more concepts accumulate, and the structural budget's dependence on total samples seen are all examples of this pattern. The intent is that the same code should behave sensibly whether it has seen a hundred examples or a hundred million, without needing its constants re-tuned by hand.

21. Honest Observations — Rough Edges in the Current Code

This section is meant to be genuinely useful rather than flattering. Reading through every file, a few patterns show up repeatedly that are worth cleaning up before you rely on this as a finished system:

- **A lot of commented-out code sits alongside the active code**, especially in cluster.py (an unused repulsion force, an unused "semantic gravity" attraction toward memory-store centres, an unused dynamic cluster-splitting block, an unused dead-cluster removal block) and in temperature.py (two abandoned resizing schemes) and budget_scheduler.py (an abandoned simpler budget formula). None of this affects behaviour today, but it makes the files noticeably harder to read, and it's easy to lose track of which version is actually live.

- **Debug print statements are still active** in concept_memory.py's **consolidate_cluster()** — several unconditional **print()** calls will spam the console every time consolidation is attempted for any cluster. Worth gating behind a logging flag or removing before any long training run.
- **debug_dataset.py** looks like a one-off utility script (it downloads the 20 Newsgroups dataset and saves it to disk) rather than a core part of the framework, and isn't imported anywhere else — it's fine to keep as a helper for testing but it is not part of the model itself.
- **Two separate energy computations exist** and it's easy to conflate them: energy_controller.py's **EnergyController.compute()** (a step-level summary, not currently used to make any decision) and structural_executor.py's internal **_compute_energy_proxy()** (a different, cheaper formula that *does* directly control accept/reject decisions during structural execution). They share the general idea but are not the same calculation, use different weights, and are not kept in sync.
- **Small duplicate logic in concept creation**: structural_system.py's concept-formation loop has a case ("CASE 2: create new concept") that duplicates logic already present just above it in the same loop, including an accidental double-increment of **new_concepts** in one branch. It doesn't appear to break anything, but it's worth simplifying into a single code path.
- **StructuralQueue.pop() rebuilds proposal keys slightly differently than push()** for non merge/split events (it uses the internal counter rather than the original center-based key), which means the duplicate-prevention guarantee described in the docstring doesn't fully hold for that event type. Since "add" events aren't currently pushed anywhere in the rest of the code, this hasn't caused a visible bug yet, but it would be worth revisiting if that event type is used in the future.
- **A few features are wired up only partially**: the memory store's **semantic_gravity()** and the cluster layer's memory-store attraction are both implemented but never actually called from the live code path (the attraction block in cluster.py is commented out); causal_consistency in the energy controller always defaults to zero, meaning that term never actually contributes anything yet.

None of these are fundamental design flaws — they read like the natural residue of active, iterative development (trying an idea, leaving it commented out rather than deleted, debugging with print statements). A focused pass to strip dead code, replace prints with proper logging, and unify the two energy formulas would meaningfully improve the codebase's readability without changing how it behaves.

22. Glossary of Terms

Embedding — A vector (a list of numbers) that represents a piece of data in a space where similar things end up close together.

Cluster — A soft, Gaussian-shaped region in the embedding space, defined by a centre, a spread, and a mass. The basic unit of the fast structural layer.

Mahalanobis distance — A distance measure that accounts for a cluster's own shape — it stretches or compresses each direction according to how spread out the cluster is along that direction, unlike plain straight-line (Euclidean) distance.

Soft assignment — Assigning a data point to every cluster with a probability, rather than picking one single "winning" cluster.

EM (Expectation-Maximization) — A classic two-step algorithm for fitting mixtures of Gaussians: estimate soft assignments (expectation), then update cluster parameters to fit those assignments better (maximization), repeating indefinitely.

Entropy — A measure of uncertainty in a probability distribution. Low entropy means the assignment is confident (mostly one cluster); high entropy means it is spread thin across many clusters.

Drift — How far a cluster's centre moved since the previous update — a proxy for instability.

Temperature — A per-cluster scalar that rises with instability/uncertainty and falls with settling down, used to gate how much structural change is allowed.

Redundancy — A persistent, slowly-accumulating measure of how similar two clusters (or, separately, two concepts) have consistently looked over time.

Structural queue — A priority queue of proposed merges/splits, with time-decayed priority and duplicate prevention.

Budget — The number of structural operations (merges/splits) the system allows itself to actually perform in one structural phase.

Energy — A single scalar combining several signals (entropy, redundancy, drift, memory pressure, coherence) where lower is considered a healthier system state.

Episodic memory — The per-cluster rolling store of recent representative example vectors, kept in memory_store.py — the system's short-to-medium-term memory.

Consolidation — The process of deciding which clusters are stable enough to be linked to permanent concepts.

Concept — A permanent, named region of long-term knowledge in concept_memory.py, distilled from one or more stable clusters over time.

Concept graph — A weighted graph recording how strongly pairs of concepts relate to each other, including automatically-formed higher-level "meta-concepts".

Spreading activation — A search technique that starts activation at one node and lets it flow outward along graph edges, decaying with distance — used by both concept_graph.propagate_activation() and the more careful priority-queue-based concept_reasoner.py.

Micro-cluster — A small, informal sub-cluster that forms in the ambiguous "grey zone" near the edge of a concept, used as an early signal that something new may be forming.

End of document.